

“Before we execute, change the plan to use localStorage instead of a database.”“I don’t like step 2, let’s rewrite that part to use a CSS animation instead.”The agent will read your feedback, update the implementation_plan.md artifact with the new approach, and pause again for your final approval. Once you are happy, say “Approved” and it will enter the “Execute” phase.

How is the agent going to debug what it has developed?The agent is entirely language and framework agnostic. It leverages pi’s native toolset to inspect your repository, figure out what stack you are using, and independently verify its work. During the “Verify” phase of the workflow, the agent will:

Use the bash tool to run your project’s native test suite (whether that’s npm run test, pytest, cargo test, go test, etc.).Run compilation checks or typecheckers specific to your language.If an error occurs, the output is returned directly to the agent’s context window. It will read the stack trace, analyze the failure, and use its editing tools to iterate and fix the bug autonomously.How can I prevent the agent from getting stuck in a loop?Occasionally, an LLM might get caught in a loop (e.g., applying the same fix repeatedly that doesn’t resolve a test failure). Here is how you manage this:

Interrupt and Steer: You are always in control. If you see the agent looping, hit Ctrl+C in your terminal to interrupt its tool execution. You can then provide a hint like “You are looping. The issue is actually in the database schema, not the route handler.”Context Compaction: pi natively compresses context. If a loop goes on too long, old context is dropped, which can sometimes break the loop organically.Interactive Grilling: If you anticipate a complex debugging session, use /grill-me beforehand to ensure the agent fully understands the edge cases before it writes code, significantly reducing the chance of cyclical debugging.



understand your caching rules without you needing to repeat

them.

4. Experiencing the Verifier Loop (Self-Correction)If the

agent makes a mistake while writing code, you might not

even notice. The harness catches it first:

Jeff (Agent):

Uses the edit tool to modify a Python file, but accidentally

introduces a syntax error.

Jeff Extension (Harness):

Intercepts the edit, runs the language linter silently in the

background. The check fails. Injects an error directly back

into the agent's context: [Harness Verifier Phase Gate Failed]

Your Python code edit failed syntax checks. You MUST fix

this error before proceeding.

Jeff (Agent):

Reads the injected error stack trace, realizes the mistake, and

issues a second edit tool call to fix the bug before finalizing

its turn.

You (User):

Only sees the final, perfectly working code, unaware that the

agent was forced to correct its own bugs in the background!

FAQ: Managing the Agent

How do I approve or amend a plan so the agent starts developing?Because the injected <planningmode> rules instruct the agent to "Obtain User Approval" before proceeding to execution, the agent will naturally stop and wait after it finishes writing the implementationplan.md. To approve it, simply type a response in the terminal like: "Looks good, go ahead and execute."If you want to amend the plan before execution: Just tell the agent what to change! Because the agent is paused waiting for your feedback, you can reply with:

Cover

Title: Agent Jeff

Author: Jeff Meridian

Title: Jeff's extension for PI

Author: Jeff Meridian

The Jeff Extension is a standalone, global extension for the pi coding agent. It transforms the default agent experience into a highly structured, self-organizing developer assistant by introducing a "Brain" (working memory) and "Knowledge Items" (long-term memory).

Why does this extension exist?

By default, an AI coding assistant starts each session with a blank slate, relying entirely on the immediate context you provide. For complex, multi-session, or architectural tasks, this can lead to:

Lost Context: The agent forgets design patterns or decisions made in previous sessions.Unstructured Execution: The agent might jump straight into modifying code without fully understanding the architecture or gaining your approval.Scattered Artifacts: Important documents like task lists and design plans are generated but not tracked persistently.Jeff solves this by introducing:

Planning Mode: Forces the LLM to pause and formulate an architectural plan (with user approval) before modifying the codebase on complex tasks.**Persistent Knowledge (KIs):** Allows the agent to read and write "Knowledge Items" (metadata.json) across sessions, ensuring it learns your specific repository patterns over time.**Structured Brain:** Automatically routes tracking documents (task.md,

You:

“Refactor the authentication flow to use JWT instead of sessions.”

Jeff (Agent):

“Because this is a major architectural change, I am entering Planning Mode. I will research the codebase first and create an `implementation_plan.md` in `~/.jeff/brain/<session-id>/` for your review.”

2. Using Slash Commands

You can manually steer the agent using the registered slash commands in the interactive terminal.

You:

`/plan` add a new Redis caching layer to the API

Jeff (Agent):

Creates an `implementation_plan.md` in your brain directory and asks for approval.

You:

`/distill`

Jeff (Agent):

Analyzes the conversation and writes a new `metadata.json` file into `~/.jeff/knowledge/redis-caching-pattern/` summarizing how caching should be handled in this specific project.

3. Next Session

The next time you launch pi, Jeff’s `beforeagentstart` hook will read the `redis-caching-pattern` KI and inject its summary into the system prompt. The agent will now natively

`implementation_plan.md`, `walkthrough.md`) into a dedicated `~/.jeff/brain/<session-id>/` directory so your project root stays clean while the agent stays organized. **Technical Details** The extension is a TypeScript module loaded dynamically by pi on startup. It lives in your global extensions directory at `~/.pi/agent/extensions/jeff.ts`.

1. Context Injection (beforeagentstart)

The extension hooks into the `beforeagentstart` event. Before the LLM generates a single token, Jeff modifies the system prompt to inject:

Workspace Metadata: Injects the current OS

(`os.platform()`), local time, and active username to give the model real-world context. **KI Summaries:** Scans `~/.jeff/knowledge/` for `metadata.json` files. If it finds previous

learnings, it injects one-line summaries of these concepts

into the prompt. **Artifact Definitions:** Tells the agent exactly where its working memory (`~/.jeff/brain/<session-id>/`) is

located for the current session. **Behavioral Directives:**

Appends a strict `<planningmode> ruleset` and a `<slashcommands>` menu to the system prompt.

2. Slash Commands

The extension registers custom interactive commands using `pi.registerCommand`, and also utilizes native commands to

- guide the agent:
- /plan [task] -> Triggers the agent to generate an implementation_plan.md in the brain directory for the specified task.
 - /distill -> Instructs the agent to extract the key learnings from the current session and write them to a new Knowledge Item.
 - /grill-me -> Prompts the agent to enter an interactive interview mode to clarify underspecified requirements before writing code.
 - /goal [objective] -> Instructs the agent to work autonomously towards a long-running goal without stopping until fully achieved.
 - /schedule [time] [prompt] -> Sets a recurring schedule or a one-time timer for the agent to run a specific instruction in the background.

3. Directory Structure

The extension automatically manages the following

directories:

~/jeff/brain/<session-id>/ - Ephemeral working memory per session (Task lists, plans, walkthroughs).~/jeff/knowledge/ - Long-term persistent memory across sessions (Design decisions, bug fixes, codebase rules).4. Harness

Orchestration (Self-Correction, JIT Reminders & BFS

Exploration)Jeff acts as an advanced orchestrator that

strictly enforces quality and context during execution:

The Verifier Loop (Dual-Layer Phase Gate): Whenever

the agent modifies code, Jeff intercepts the tool output. First,

it searches for a .py .sh convention script to run

custom project checks. If not found, it falls back to zero-

config language linters (like ruff for Python or tidy for

HTML). If any check fails, the logs are bounced directly

back to the agent as a hard error, forcing it to self-correct and fix the bugs before continuing.**Just-In-Time (JIT) Context Reminders:** Rather than overloading the system prompt with massive rulebooks, Jeff dynamically injects targeted micro-reminders into the agent's attention window precisely when they are needed. For example, when editing a .ts file, the agent receives an immediate injected reminder to avoid inline imports and single-line helper functions.**Task-Specific Model Routing (SLM Delegation):** Jeff dynamically switches the active LLM based on the task type. Complex tasks (coding, debugging) use your primary heavy model, while simpler tasks (like the /plan or /distill slash commands) are automatically routed to faster, cheaper Small Language Models (SLMs) configured in jeffmodelrouting.md.**Code Graph BFS Exploration:** Jeff fundamentally alters how the agent explores the codebase by injecting the "Codebase Archaeologist" constraint. The agent is strictly forbidden from reading entire files over 50 lines linearly. Instead, it must trace dependencies using line-level slicing and document its hypothesis internally before executing reads.get_outline Tool: Jeff registers a custom tool that acts as a structural parser. It allows the agent to extract the skeleton of a file (classes, functions, interfaces) without polluting its context window with the source code, enabling surgically precise reads.**Example Usage**Once installed in ~/pi/agent/extensions/jeff.ts, the extension runs automatically every time you start pi.

1. Initiating a Complex Task

If you ask pi to implement a major feature, the injected Planning Mode rules will stop it from immediately writing code. Instead, it will output a plan.